Transformer-Based Masked Autoencoder With Contrastive Loss for Hyperspectral Image Classification

Xianghai Cao, *Member, IEEE*, Haifeng Lin, Shuaixu Guo, Tao Xiong, *Member, IEEE*, and Licheng Jiao, *Fellow, IEEE*

***Abstract*— In recent years, in order to solve the problem of lacking accurately labeled hyperspectral image data, self-supervised learning has become an effective method for hyperspectral image classification. The core idea of self-supervised learning is to define a pretext task, which helps to train the model without the labels. By exploiting both the information of the labeled and unlabeled samples, self-supervised learning shows enormous potential to handle many different tasks in the field of hyperspectral image processing. Among the vast amount of self-supervised methods, contrastive learning (CL) and masked autoencoder (MAE) are well known because of their impressive performance. This article proposes a Transformer-based MAE using contrastive learning (TMAC), which tries to combine these two methods and improve the performance further. TMAC has two branches. The first branch has an encoder–decoders structure, and it has an encoder to capture the latent image representation of the masked hyperspectral image and two decoders where the pixel decoder aims to reconstruct the hyperspectral image at pixel-level and the feature decoder is built to extract the high-level feature of the reconstructed image. The second branch consists of a momentum encoder and a standard projection head to embed the image into the feature space. Then, we combine the output of feature decoder and the embedding vectors via CL to enhance the model's classification performance. According to the experiments, our model shows powerful feature extraction capability and gets outstanding results on hyperspectral image datasets.**

***Index Terms*— Classification, contrastive learning (CL), hyperspectral imagery (HSI), masked autoencoder (MAE), self-supervised learning.**

I. Introduction

HYPERSPECTRAL imaging has become an increasingly popular technique for remote sensing and Earth observation applications in recent years. This is due to its ability to capture data across many contiguous spectral bands [1], allowing for the identification and analysis of land cover materials with high accuracy. The hyperspectral imagery (HSI) can be applied to many fields, such as precision agriculture [2], military application [3], and food safety inspection [4]. One of the key missions in hyperspectral image analysis is the classification of the land cover types presented in the scene. The high dimensionality of hyperspectral data, along with the presence of noise and variability in the data, poses significant challenges for effective classification. Traditional classification methods, such as random forests, support vector machines, sparse learning, and Markov random fields, have been widely used in the HSI fields [5], [6], [7], [8]. Many feature extraction and dimension reduction methods are also proposed to improve the classification performance, such as principal component analysis (PCA) [9], independent component analysis [10], and extinction profiles [11]. These methods mentioned above can certainly get great outcomes in small hyperspectral datasets; however, as the amount of data increases, due to the limited feature extraction capability and small model parameters of these handcrafted methods, they cannot handle the complexity and heterogeneity of hyperspectral data effectively.

In recent years, deep learning techniques, such as convolutional neural networks (CNNs) [12], [13], have emerged as powerful tools for hyperspectral image classification. These methods can automatically learn hierarchical and discriminating features from the input data by their powerful nonlinear learning ability and have shown promising results in various applications [14]. The main idea of CNN is an operation called convolution, which captures the local information of the image. Based on the CNN module, Lee and Kwon [12] proposed contextual CNN by jointly exploiting local spatial–spectral relationships of neighboring pixels, Yu et al. [15] tried to combine 2D-CNN with 3D-CNN for classification. By introducing the residual block, Zhong et al. [16] proposed spectral–spatial residual network (SSRN) to alleviate the declining-accuracy phenomenon. In [17] and [18], recurrent neural networks (RNNs) were presented to handle the HSI data sequentially. In [19], the graph convolutional networks were proposed to model the relations between the samples. In [20] and [21], a multimodal deep learning method is introduced to handle the more complicated spectral data. However, the local receptive field in CNNs has limits in modeling long-range dependencies in an image [22]. Recently, the Transformer is introduced for HSI classification because it can capture the long-range dependencies of spectral information [23], [24]. In [25], a spatial–spectral transformer with cross-attention method is proposed to extract the interactive spatial–spectral fusion feature. Tu et al. [26] proposed a local semantic feature aggregation-based Transformer for hyperspectral image classification.

On the other hand, all the models mentioned above are supervised models, which means that the training of these

Manuscript received 26 April 2023; revised 28 August 2023; accepted 12 September 2023. Date of publication 15 September 2023; date of current version 28 September 2023. This work was supported by the National Natural Science Foundation of China under Grant 62176199. *(Corresponding author: Haifeng Lin.)*

The authors are with the School of Artificial Intelligence, Xidian University, Xi'an 710071, China (e-mail: georgelin0825@gmail.com).

Digital Object Identifier 10.1109/TGRS.2023.3315678

1558-0644 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 07:21:48 UTC from IEEE Xplore. Restrictions apply.

models needs vast amount of accurately labeled samples [27]. Though unlabeled hyperspectral data are easy to collect, accurate label information is still hard to acquire. To solve this problem, semisupervised learning has been proposed. The main idea of semisupervised learning is using unlabeled data together with the labeled data for model training. In past decades, a great number of semisupervised methods have been proposed [28]. Wu et al. [29] proposed a semisupervised model, which used self-training to assign pseudo labels to unlabeled samples and introduced a spatial constraint to regulate the pseudo labels. Liu et al. [30] combined CNN with semisupervised learning to solve the problem of limited labeled samples. In [31], a generative adversarial network was improved for HSI classification. Although semisupervised learning could alleviate the limited labeled samples problem to some extent, they often need to select appropriate hyperparameters and the inaccurate pseudo labels may deteriorate the model performance. Besides, transfer learning and few-shot learning are also used to reduce the need for labeled samples [32], [33].

Recently, self-supervised learning has attracted much attention because of its excellent performance. The core idea of self-supervised learning is to train the model with pretasks where the label can be obtained directly based on data characteristics. For now, two representative self-supervised learning methods are contrastive learning (CL) and masked autoencoder (MAE). CL is able to produce impressive transferable visual representations by learning to be invariant to different data augmentations [34]. The MAE represents a different idea to learn the image representation by masking the origin image and forces the model to reconstruct the image [35]. Because these two methods are designed with totally different ideas, we think that they may extract complementary information from the data and can be combined to obtain more powerful features. Though these two methods have been introduced for HSI classification separately [36], [37]. In this article, we present a stronger model by combining the power of mask autoencoder and CL and adopt the Transformer networks as the backbone for the whole model. The proposed model consists of two branches: the first branch is used to reconstruct the masked hyperspectral image to learn the pixel-level information, the second branch has the same encoder structure as the first branch, but the input is not masked, and its projection output is combined with the embedding feature of the first branch to compute the contrastive loss.

The main contributions of this article are summarized as follows.

1) A new Transformer-based network—Transformer-based masked autoencoder using contrastive learning (TMAC)—is proposed as a feature extractor for HSI classification. The proposed model can exploit the complement power of CL and MAE.
2) The TMAC model uses MAE to learn the pixelwise feature and uses CL to obtain the global feature.
3) The TMAC method shows better classification performance compared with CL- and MAE-based models.

The remaining parts of this article are listed as follows. Section II introduces details of some related works. Section III describes the details of the proposed model. Section IV shows the performance of the proposed model by using three popular datasets. Finally, Section V gives a conclusion of our work.

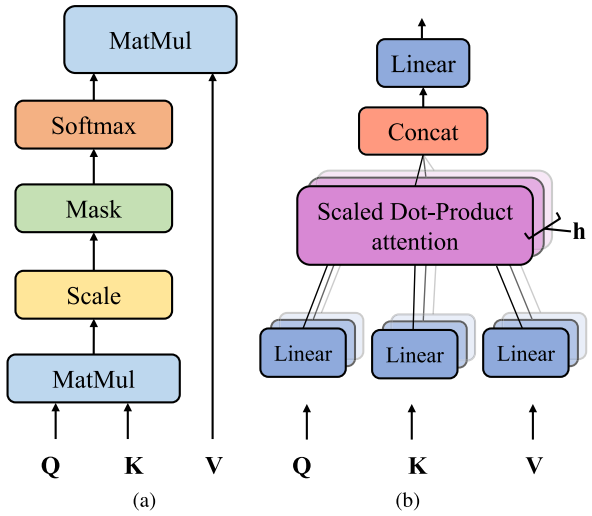


Fig. 1. Illustration of the attention mechanism. (a) SA. (b) MSA.

II. Related Work

A. Transformer and Vision Transformer

Transformer is a kind of deep learning model that has become increasingly popular in recent years. Originally introduced by Vaswani et al. [38], Transformer has become the most widely used architecture in natural language processing (NLP) and other machine learning tasks. Transformer is originally designed to deal with the sequences of data. Unlike traditional RNNs, which process data sequentially, Transformer processes all elements of a sequence in parallel. This parallel processing makes Transformer faster and more efficient than RNNs and has contributed to its widespread applications. Transformer is particularly well suited to NLP tasks because it can capture long-term dependencies in text, making them effective at machine translation and sentiment analysis. They do this through the use of self-attention (SA) mechanisms, which allow the model to weight different parts of the input sequence differently depending on their importance.

SA uses three vector matrices W_q, W_k, and W_v to multiply the input embedding and get the query Q, the key K, and the value V. The attention score can be calculated by the inner product of Q and K, and the score will be applied normalization, which helps to stabilize the gradient; then, a softmax function will be used for the score and get the final representation by multiplying the V vector matrix. To sum up the whole process, the SA function could be formulated as

$$\text{SA} = \text{attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V. \quad (1)$$

The computation of SA is also shown in Fig. 1(a), where d is the dimension of matrix K. The multihead attention (MSA) mechanism includes multiple blocks and each of them is composed of various groups of the Q, K, and V matrices. As shown in Fig. 1(b), the same SA layer is used to calculate

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 07:21:48 UTC from IEEE Xplore. Restrictions apply.

the attention score and all groups are concatenated. The process can be described as

$$\mathrm{MSA}(Q, K, V) = \mathrm{Concat}(\mathrm{SA}_1, \mathrm{SA}_2, \ldots, \mathrm{SA}_h)W^l \quad (2)$$

where h is the number of heads and W^l means the learnable parameter matrix.

Vision Transformer (ViT) is an improved version that applies the Transformer architecture to image classification tasks [39]. Unlike NLP's 1-D sequence data, the data in computer vision are 2-D or 3-D, and the semantic density of image is much sparser compared with text. Thus, ViT divides an input image into a set of fixed-size nonoverlapping patches, which serves as the input to the subsequent Transformer layers. After patch embedding, ViT uses SA mechanisms to process image patches as a sequence, allowing them to capture both local information and global information. This is different from traditional CNNs, which process images in a hierarchical manner and may miss some important information. This is really important for hyperspectral images because they have hundreds of bands, which demand the model to focus on both spatial information and spectral information.

In recent years, many Transformer-based works have been proposed. He et al. [23] used traditional CNN to capture the spatial features and input them to a modified Transformer block to get the sequential spectral relationships. Qing et al. [40] proposed SAT by combining spectral attention modules and Transformer blocks to extract spatial–spectral features. Sun et al. [41] combined Transformer and a Gaussian weighted feature tokenizer to improve the performance of Transformer-based networks. Also, Hong et al. [42] proposed a special Transformer network called SpecTransformer, which focuses on the local sequence information inside the hundreds of spectral bands.

B. Contrastive Learning

Recently, CL has gotten quite attention in self-supervised learning fields. The main pretext task of CL is very simple but efficient. Consider that we have an image $x \in \mathbb{R}^{H\times W\times C}$ as anchor, and we refer its augmented view $x^+ \in \mathbb{R}^{H\times W\times C}$ as a positive sample, which means that x^+ is similar to the image x, and we also need some negative samples $x^- \in \mathbb{R}^{H\times W\times C}$, which can be constructed with samples except x. Next, the images are projected into embedding space, and then, the model is trained to make the positive pairs as close as possible and the negative pairs as far as possible. The main challenge of CL is how to define the positive and negative samples. In [43], a classic paradigm for CL was proposed, in which data augmentation is used to make positive samples and momentum update is used to form the negative samples. Also, in [44], by increasing the batch size and adjusting the model structure, its performance almost overtakes the supervised methods. With only the positive samples, Grill et al. [45] also showed high performance. Another important thing is the loss function. Different from supervised learning, cross-entropy loss is not suit for the CL. Chen et al. [44] proposed a loss function called infoNCE

$$L_{\mathrm{info}} = -\log\frac{\exp\left(\mathrm{sim}\left(x, x^+\right)/\tau\right)}{\sum_{n=1}^{N}\exp(\mathrm{sim}(x, x^-)/\tau)} \quad (3)$$

where N means the number of images in a batch and τ means an adjustable temperature coefficient. The $\mathrm{sim}(\cdot)$ function is defined as

$$\mathrm{sim}(u, v) = \frac{u^T v}{\|u\| \cdot \|v\|}. \quad (4)$$

As τ goes up, the distribution becomes smooth, and as the value goes down, the distribution will concentrate and form more peaks. The main goal of the loss function is the same as the idea mentioned above: to bring the positive pairs closer and push the negative pairs farther away.

C. Mask Image Modeling

Mask image modeling (MLM) is a new pretext task in self-supervised learning. This generative pretraining paradigm is first used in NLP [46] by masking some words and forces the model to generate the masked words. In recent years, researchers have tried to introduce this idea to the image field [47]. Due to the genius strategy of ViT method that considers an image block as a word, it is easy to transfer the generative pretraining paradigm to images. He et al. [35] proposed MAE that masked the image in a pretty high ratio and forced the model to reconstruct the masked patches in pixelwise. Bao et al. [48] first trained a tokenizer module to discretize the image patches and then encode the masked image patches into a discrete matrix and reconstruct the masked parts. Ibanez et al. [37] introduced the masked image modeling method to HSI classification.

III. Proposed Model

In this section, we will introduce the details of the proposed model TMAC. First, we will introduce the data preprocessing for hyperspectral images. Next, the model TMAC will be shown in detail, which is mainly composed of two parts: MAE branch and CL branch. Finally, we also show how to use TMAC for the HSI classification. Fig. 2 shows the overall framework of TMAC.

A. Preprocessing

The HSI data often have a great amount of bands, which results in a dramatic increase in computation. Thus, it is necessary to reduce the dimensionality of HSI data. We choose to use the PCA for this part. PCA is a widely used technique in hyperspectral image classification for reducing the dimensionality of the data while preserving the most important information. By projecting data into a low-dimensional subspace that captures the most significant information in the data, PCA can overcome the curse of dimension and improve the accuracy and efficiency of classification.

For the original HSI data, $I \in \mathbb{R}^{H\times W\times C}$, which is reduced to $I_{\mathrm{pca}} \in \mathbb{R}^{H\times W\times b}$ after PCA. For TMAC, we reduced the dimension to 9 after the PCA processing. Different from image classification, HSI classification needs to classify every pixel in image, so we used the cube around the target pixel as the feature; then, we have $C = [x_1, x_2, \ldots, x_N] \in \mathbb{R}^{s\times s\times b}$, where N is the number of extracted data cube and s is the cube size.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 07:21:48 UTC from IEEE Xplore. Restrictions apply.

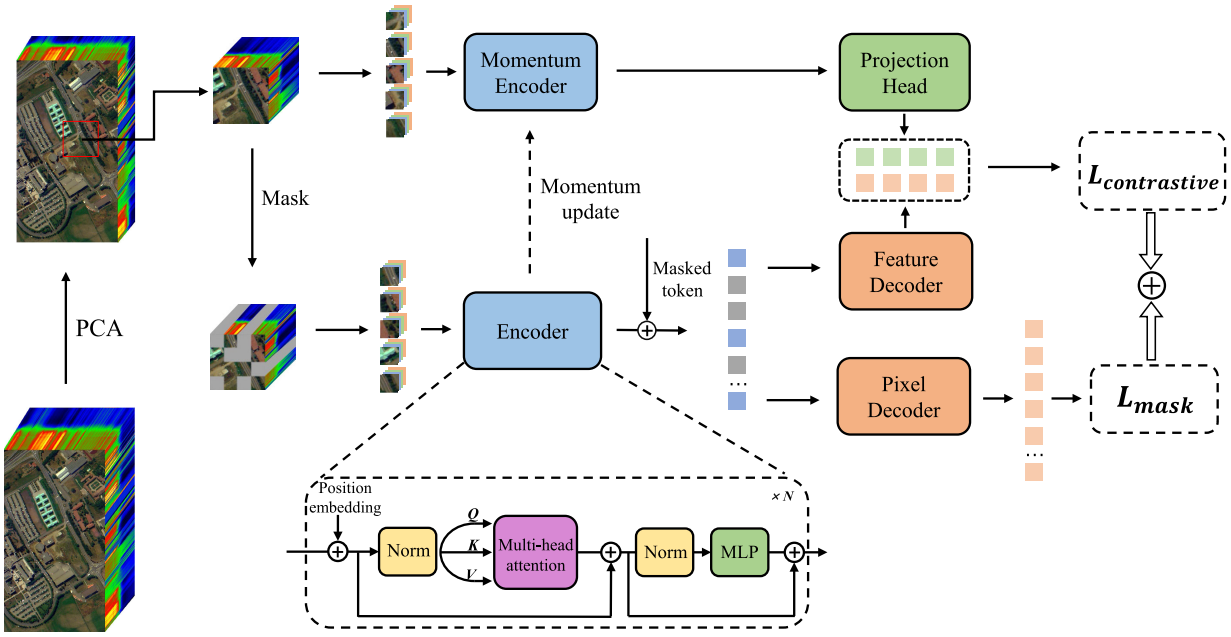


Fig. 2. Flowchart of the proposed TMAC.

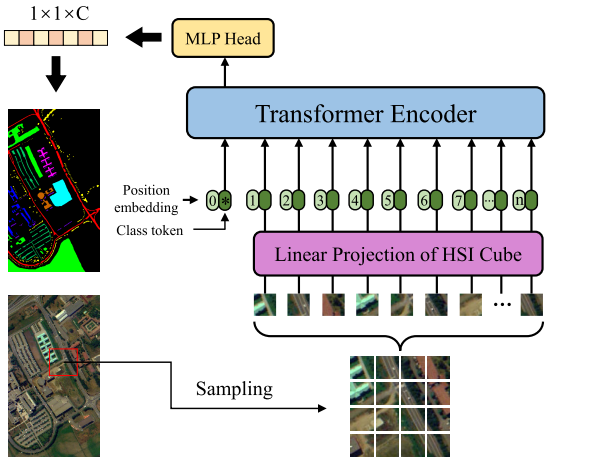
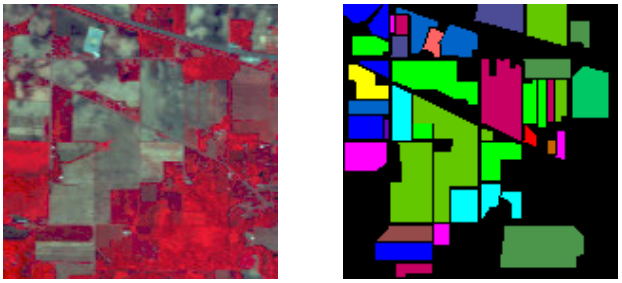


Fig. 3. Flowchart of classification stage (Transformer encoder's parameters are retained from the first stage).

TABLE I
NUMBER OF TRAINING AND TEST SAMPLES OF THE IP DATASET

Class No.	Class name	Training number	Test number
1	Alfalfa	2	44
2	Corn-notill	71	1357
3	Corn-mintill	41	789
4	Corn	12	225
5	Grass-pasture	24	459
6	Grass-trees	37	693
7	Grass-pasture-mowed	1	27
8	Hay-windrowed	24	454
9	Oats	1	19
10	Soybean-notill	49	923
11	Soybean-mintill	123	2332
12	Soybean-clean	30	563
13	Wheat	10	195
14	Woods	63	1202
15	Buildings-Grass-Trees-Drives	19	367
16	Stone-Steel-Towers	5	88
	Total	512	9737



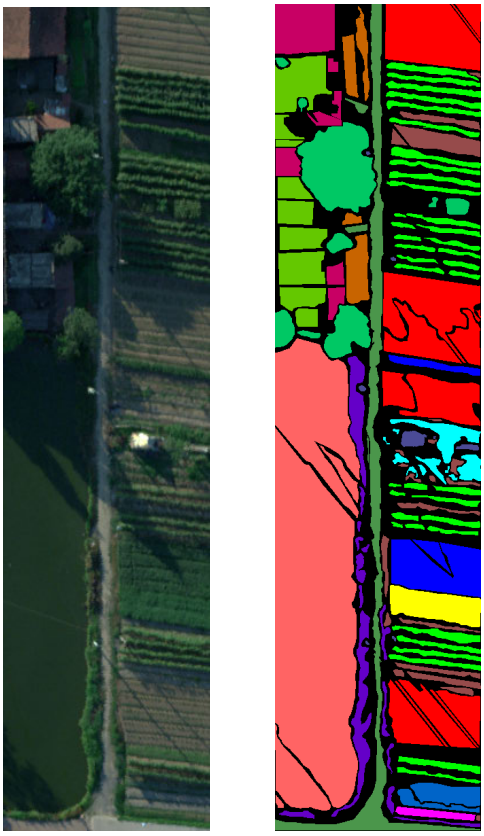
(a) (b)

Fig. 4. IP dataset. (a) False-color map. (b) Ground truth.

B. Transformer-Based Masked Autoencoder Using Contrastive Learning

1) MAE Branch: The MAE branch aims to learn the pixelwise representation of the HSI data. This branch consists of three main blocks. The first one is a Transformer-based encoder to get the latent representation of the unmasked patches. In the traditional autoencoder structure, a neural network is used to adaptively learn the representation of the origin HSI data. However, with the model scale-up, this method cannot exploit the full power of the neural network as it just simply learned the shallow HSI representation [49]. Thus, in this work, we mask the origin sample to destroy the obvious image features, which will force the encoder to learn the deep image representation. For the HSI data cube $x \in \mathbb{R}^{s\times s\times b}$, we use an $m\%$ ratio to randomly mask the data cube. To be more precise, a patch embedding will be used for data cube x in a ViT way. We have $p \in \mathbb{R}^{N\times P^2\times b}$ after patch embedding where P is the patch size and $N = (s \times s)/P^2$ is the number of patches. After patch embedding, the position embedding will be added to the patches, which contains the location information of different patches in the origin cube.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 07:21:48 UTC from IEEE Xplore. Restrictions apply.



(a) (b)

Fig. 5. HC dataset. (a) False-color map. (b) Ground truth.

TABLE II
NUMBER OF TRAINING AND TEST SAMPLES OF THE HC DATASET

Class No.	Class name	Training number	Test number
1	Strawberry	224	44511
2	Cowpea	114	22639
3	Soybean	51	10236
4	Sorghum	27	5326
5	Water spinach	6	1194
6	Watermelon	23	4510
7	Greens	29	5874
8	Trees	90	17888
9	Grass	47	9422
10	Red roof	52	10464
11	Gray roof	84	16827
12	Plastic	18	3661
13	Bare soil	46	9070
14	Road	93	18467
15	Bright object	6	1130
16	Water	377	75024
	Total	1287	256243

Then, the mask operation will be executed as

$$I_m = M(p + p_e) \tag{5}$$

where p_e means the position embedding and the $M()$ function means the mask operation, which will mask $m\%$ of the patches, and the remaining $(1 - m\%) \times N$ patches will be used as the input of Transformer-based encoder E_p. After encoding the masked patches, the output will be reshaped to the original size by padding blank on the masked fields, and then, a pixel decoder D_p is used to reconstruct the masked part of the origin HSI data pixelwisely. The decoder is also composed of Transformer blocks. Finally, a linear layer is used to predict the masked part of the image. As the loss function, an L_2 loss is adopted here

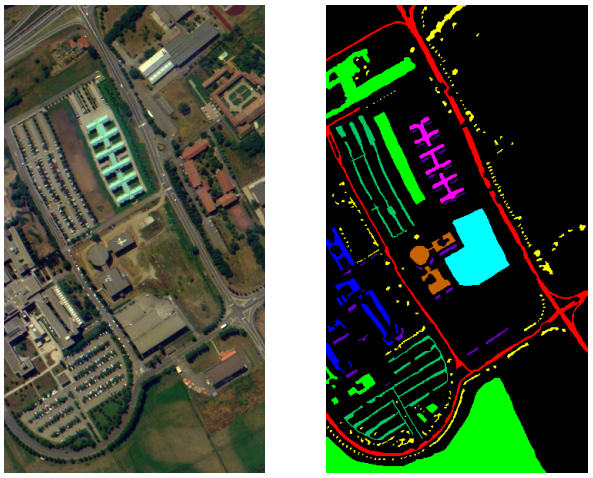
$$L_{\text{mask}} = \frac{1}{K} \sum_{n=1}^{K} \left(k_p - \hat{k_p}\right)^2 \tag{6}$$

where K means the number of the masked patches, k_p is the true value of the origin patch, and $\hat{k_p}$ is the predicted patch.

By the way, another feature decoder D_f is also used to get the high-dimensional feature of the HSI data, and its output k_f will be combined with contrastive branch for CL.

2) CL Branch: Different from the MAE branch, the CL branch focuses on catching the high-level feature representation. Its purpose is to force the encoder to not only learn the pixelwise feature but also the holistic feature. The feature encoder of CL branch is almost the same as the pixelwise branch, but the input cube is unmasked, which could help the encoder to focus on the holistic feature. After the input patches are fed into the feature encoder E_f, which has the same structure as the encoder E_p, we use a mean-pooled function to get the global spatial–spectral feature

$$h = \frac{1}{N} \sum_{i=1}^{N} E_f(x_i) \tag{7}$$



(a) (b)

Fig. 6. PU dataset. (a) False-color map. (b) Ground truth.

TABLE III
NUMBER OF TRAINING AND TEST SAMPLES OF PU

Class No.	Class name	Training number	Test number
1	Asphalt	66	6565
2	Meadows	186	18463
3	Gravel	21	2078
4	Trees	31	3033
5	Painted metal sheets	13	1332
6	Bare Soil	50	4979
7	Bitumen	13	1317
8	Self-Blocking Bricks	37	3645
9	Shadows	10	937
	Total	512	9737

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 07:21:48 UTC from IEEE Xplore. Restrictions apply.

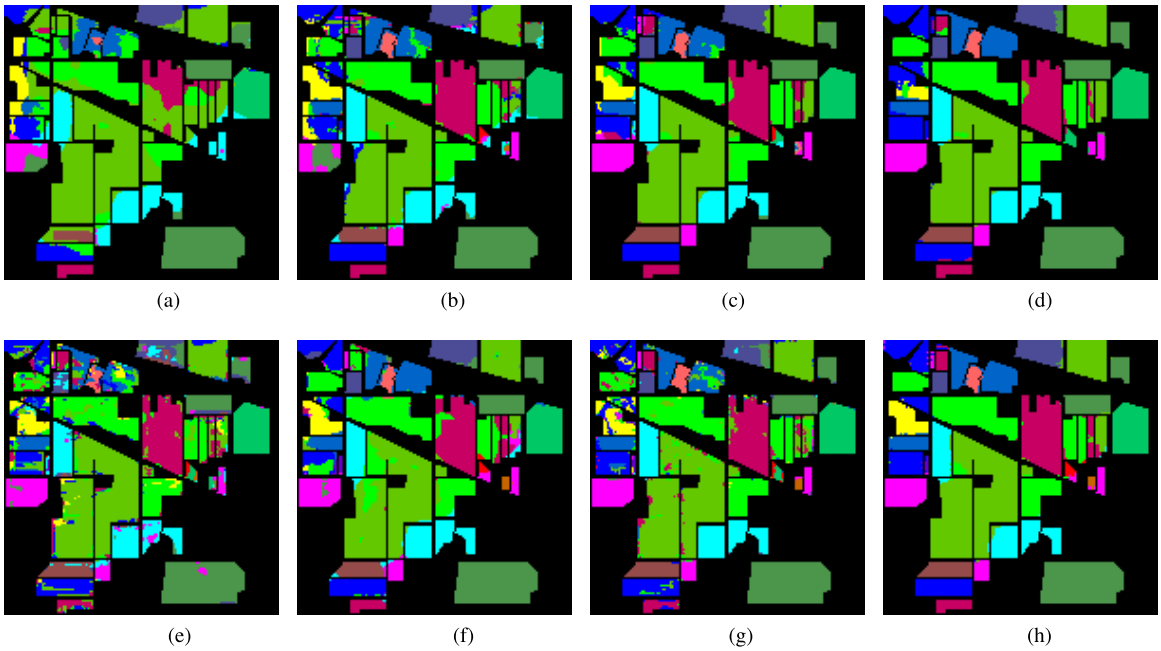


Fig. 7. Classification maps of IP. (a) SVM. (b) ResNet. (c) HybridSN. (d) DBDA. (e) SF. (f) CLB. (g) MAEST. (h) TMAC.

TABLE IV
CLASSIFICATION PERFORMANCE OF DIFFERENT METHODS ON THE IP AND THE BEST ONE IS SHOWN IN BOLD

No.	SVM	ResNet	HybridSN	DBDA	SF	CLB	MAEST	TMAC
1	0.00	52.27	25.00	0.00	25.00	43.48	17.39	**63.64**
2	62.34	92.56	92.70	94.25	78.11	91.81	86.20	**95.50**
3	56.02	87.71	84.41	94.80	64.38	78.80	90.48	**98.10**
4	49.33	78.22	72.44	33.78	65.78	83.12	78.06	**94.67**
5	48.37	73.64	**99.78**	94.77	85.40	94.62	93.79	98.47
6	90.76	95.82	96.54	99.13	88.31	98.22	**99.04**	95.96
7	0.00	37.04	15.00	0.00	11.11	**100.00**	21.43	77.78
8	93.83	97.80	**100.00**	**100.00**	99.56	98.74	95.82	**100.00**
9	0.00	10.53	11.36	0.00	31.58	**90.00**	65.00	68.42
10	51.03	89.92	92.96	90.90	84.83	84.67	82.30	**98.27**
11	94.90	93.27	97.51	99.53	84.99	88.84	86.97	**98.16**
12	52.40	79.40	**97.51**	96.09	53.64	88.20	70.99	96.63
13	52.82	84.62	**100.00**	98.46	90.25	90.24	99.51	**100.00**
14	98.67	98.67	99.75	**100.00**	91.76	**100.00**	99.45	99.50
15	66.49	83.65	89.65	87.19	49.04	93.52	80.83	**95.91**
16	15.91	71.59	72.73	87.50	57.95	73.12	**97.85**	82.95
OA(%)	73.96	90.15	93.11	94.19	79.79	90.55	88.29	**97.28**
AA(%)	52.05	76.67	77.93	73.53	66.36	87.34	79.07	**91.50**
κ	69.51	88.76	92.11	93.34	76.89	89.24	86.66	**96.89**

where x_i means the patch representation of input. After the encoder, a two-layer projection network is adopted to project the representation to feature embedding space, we have the holistic feature h_f of input data. Then, the output of feature decoder in the pixelwise branch is combined with it to compute the contrastive loss as

$$L_{\text{contrastive}} = -\log \frac{\text{con}\left(k_f, h_f^+\right)}{\text{con}\left(k_f, h_f^+\right) + \sum_{n=1}^{B-1} \text{con}\left(k_f, h_f^-\right)} \tag{8}$$

where B means the batch size and the $\text{con}(\cdot)$ function is defined as

$$\text{con}(a, b) = \exp(\text{sim}(a, b)/\tau) \tag{9}$$

where τ is an adjustable temperature coefficient and $\text{sim}(\cdot)$ is defined in (4). In particular, for the parameters in feature encoder E_f, we update them in an exponential moving average (EMA) way, that is, the parameter θ_f is updated as

$$\theta_f = n\theta_f + (1 - n)\theta_p \tag{10}$$

where θ_p is the parameters of E_p and $n \in (0, 1)$ is the momentum coefficient [50], which is used to prevent collapse during the training and could also make the update more smoothly. For the value of momentum coefficient, a large momentum (like 0.999) works better than a small value (like 0.9) according to the paper MOCO [43], and in TMAC,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 07:21:48 UTC from IEEE Xplore. Restrictions apply.

TABLE V
CLASSIFICATION PERFORMANCE OF DIFFERENT METHODS ON THE HC DATASET AND THE BEST ONE IS SHOWN IN BOLD

No.	SVM	ResNet	HybridSN	DBDA	SF	CLB	MAEST	TMAC
1	77.28	85.21	86.38	96.82	88.94	93.13	90.75	**99.17**
2	80.89	93.28	87.97	95.98	87.37	95.62	81.74	**96.93**
3	71.83	63.99	92.84	93.41	79.14	81.82	84.61	**93.67**
4	34.93	87.75	96.38	**97.74**	90.98	81.15	90.96	88.70
5	0.00	78.33	**93.50**	58.67	34.25	91.25	43.42	91.21
6	0.00	43.75	18.35	67.11	15.84	49.28	42.16	**68.14**
7	0.00	86.80	33.86	88.43	84.28	**98.19**	72.86	84.98
8	70.25	81.75	42.66	89.58	69.13	89.64	68.66	**94.44**
9	38.47	85.25	69.86	90.21	67.86	86.43	66.63	**94.33**
10	27.62	50.89	72.12	**95.10**	70.98	60.43	75.74	87.56
11	0.22	10.41	87.63	**98.43**	62.64	76.15	83.32	93.14
12	0.00	64.37	24.19	63.77	15.41	66.65	19.68	**84.65**
13	23.43	67.71	60.34	68.47	47.14	59.99	42.85	**84.79**
14	33.74	88.02	73.74	79.11	77.98	92.37	78.07	**96.91**
15	13.64	37.85	30.02	65.14	46.92	61.71	45.60	**85.04**
16	99.57	99.68	**99.93**	99.47	99.10	98.25	98.06	99.53
OA(%)	64.10	81.16	81.46	92.91	81.65	88.80	83.01	**95.41**
AA(%)	35.74	70.31	66.86	84.21	64.87	80.13	67.82	**90.20**
κ	54.95	77.45	77.98	91.69	78.33	86.79	80.09	**94.62**

TABLE VI
CLASSIFICATION PERFORMANCE OF DIFFERENT METHODS ON THE PU AND THE BEST ONE IS SHOWN IN BOLD

No.	SVM	ResNet	HybridSN	DBDA	SF	CLB	MAEST	TMAC
1	95.63	87.42	97.87	**98.55**	86.81	89.79	92.01	97.59
2	97.53	98.35	99.97	**99.99**	98.28	99.21	98.54	99.75
3	12.90	61.45	53.08	51.64	50.14	79.09	63.27	**85.03**
4	54.57	91.06	90.44	91.20	90.97	**94.61**	94.06	93.83
5	91.52	89.41	98.72	99.62	99.10	94.65	**100.00**	95.20
6	31.67	87.81	**97.61**	95.12	55.87	70.67	70.89	96.45
7	17.69	65.68	16.99	42.37	69.40	83.08	39.77	**99.16**
8	82.58	95.09	90.32	91.93	80.58	93.73	73.55	**97.94**
9	21.99	90.93	99.89	**100.00**	97.54	95.04	98.31	69.48
OA(%)	76.63	91.34	92.40	93.70	86.22	91.87	88.29	**96.89**
AA(%)	56.22	85.24	82.76	85.60	80.97	88.87	81.06	**92.71**
κ	67.16	88.47	89.84	91.57	81.33	89.03	84.24	**95.87**

we choose 0.99 as our momentum coefficient. The total self-supervised loss of TMAC can be computed by combining pixelwise reconstruction loss and contrastive loss as

$$\text{Loss} = L_{\text{mask}} + \lambda_f * L_{\text{contrastive}} \tag{11}$$

and λ_f is chosen as 0.5 based on the experiment result, which is shown in Table VIII.

C. Classification

This section is used for HSI classification tasks. In the previous part, the TMAC model is trained to catch the pixelwise and holistic spatial–spectral feature of the HSI data. Also, in this section, TMAC will be used as the feature extractor to classify the HSI data. In this fine-tuning stage, the input is the complete HSI data cube without masking. The backbone is the same as the pixelwise encoder E_p introduced before and the learned parameters are used to be the initialization parameter. After the encoder, an multilayer perceptron (MLP) head is used for classification and a cross-entropy loss is chosen as the loss function. Because of the powerful feature extracting ability of the model, it only takes few training epochs for the model to get a good performance. Fig. 3 shows the flowchart of the classification part.

D. Overall Algorithm

The whole classification model could be described as two stages. In the first stage, we used the unlabeled data for pretraining by combining the masked image modeling and CL. We used the two-branch structure introduced above to force the model to learn pixelwise information and holistic spatial–spectral feature of the HSI data, which produce a spatial–spectral feature extractor for hyperspectral data. Also, in the second stage, a small set of labeled samples were used to

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 07:21:48 UTC from IEEE Xplore. Restrictions apply.

fine-tune the feature extractor for the downstream classification task. The detailed steps are described in Algorithm 1.

Algorithm 1 TMAC: PyTorch-Like Pseudocode

```
# E_p/E_f: encoder/momentum encoder
# D_p/D_f: pixel decoder/feature decoder
# Proj: projection head
# λ_f: feature loss weights
# n: momentum coefficient
imgs = cube(applyPCA(data)) # data preprocessing
for img in loader: # load a minibatch
  x = patch_emb(img) # embed patches
  x_v, x_m = mask(x) # ramdom split visible and masked
    patches
  q_p, q_f = E_p.forward(x), E_f.forward(x_v)
  k_p, k_f = D_p(q_p), D_f(q_p)
  h_f = Proj(q_f)
  # compute total loss
  loss = λ_f * contrastive_loss(k_f, h_f) + mask_loss(k_p,
    x_m)
  loss.backward()
  update(E_p.params)
  # momentum update
  E_f.params = n * E_f.params + (1-n) * E_p.params
# finetune
for img, label in loader:
  x = patch_emb(img)
  # load parameters from pretrain
  encoder.load_model(E_p)
  predict = MLP(encoder(x))
  loss = CrossEntropyLoss(predict, label)
  loss.backward()
```

IV. Experiments

To verify the performance of the proposed method, we will compare the TMAC model with some classical baseline and state-of-the-art (SOTA) methods. We choose three widely used datasets: Indian Pines (IP), HanChuan (HC), and Pavia University (PU).

A. Datasets

1) Indian Pines: The IP dataset was collected by the AVIRIS sensor over the IP test site in Northwestern Indiana. The image consists of 145 × 145 pixels and has 224 spectral bands in the wavelength range of 400–2500 nm with a 10-nm spectral resolution. After removing the noisy and water absorption bands, there are 200 bands selected for test. The ground truth totally has 16 classes, which is shown in Table I. The false-color and ground-truth maps of IP are shown in Fig. 4.

2) HanChuan: The WHU-Hi-HanChuan dataset [51] was acquired in Hanchuan, Hubei province, China, with a 17-mm focal length Headwall Nano-Hyperspectral imaging sensor equipped on a Leica Aibot X6 UAV V1 platform [52]. The HC dataset consists of 1027 × 303 samples. HC contains 274 bands in the wavelength range of 400–1000 nm and the spatial resolution of the UAV-borne HSI is about 0.109 m. The ground-truth map has 16 classes and we use 0.5% randomly sampled data for training and the remaining for test. Fig. 5 shows the false-color and ground-truth map of HC dataset. Table II shows the detailed information of 16 classes.

3) Pavia University: The PU dataset was acquired by the Reflective Optics System Imaging Spectrometer (ROSIS) sensor during a flight campaign over Pavia, Northern Italy. PU covered 610 × 340 samples. The wavelength range is 430–860 nm with a spatial resolution of 1.3 m. After discarding some useless bands, there are 103 bands for testing. The PU dataset totally has nine classes and we use 1% randomly sampled data for training, which is shown in Table III, and the false-color map and ground-truth map is shown in Fig. 6.

B. Experimental Setup

To implement the proposed TMAC, we use the PyTorch framework on a Ubuntu 20.04 machine and all experiments were executed with Intel[1] Core[2] i7-9700F CPU and an NVIDIA GeForce RTX 2070 SUPER 8-GB GPU. To train the TMAC model, we use 50% samples for the self-supervised pretraining stage, and for the classify stage, different sample number is chosen for different datasets. We use 5% randomly sampled data for the IP dataset, 0.5% for the HC dataset, and 1% for the PU dataset; the detailed number of each category is shown in Tables I–III. For data preprocessing, the HSI cube size is 27 × 27 and the patch size of Transformer block is 3 × 3, which means that every HSI cube will be divided into 81 patches. For the pretraining stage, the training epoch is 300 with the batch size set to 64 and the Adam optimizer is used.

C. Comparison Methods

To show the performance of the proposed TMAC model, we choose several classic or SOTA methods for comparison. We choose SVM [6], ResNet [53], HybridSN [54], and DBDA [55], which are the classic HSI classification methods. We also choose spectral-former (SF) [42], MAEST [37], and CLB [56] to compare, where SF is a Transformer-based method, MAEST is a mask autoencoder-based method, and CLB is a CL method. To make the comparison fair enough, the training sample number of all these methods is the same as the TMAC.

Table IV shows the results of all the methods mentioned above on the IP dataset. We listed three main metrics as overall accuracy (OA), averaged accuracy (AA), and Kappa for comparison and we also listed every category's accuracy to show the detailed classification ability of different models. It can be seen that the proposed model TMAC shows the best results on all three classification indexes. For different categories, TMAC also gets the best performance for most of them. Especially for the class Alfalfa, Grass-pasture-mowed, and Oats, very few labeled samples are used for fine-tuning. It can be seen that traditional ML- and CNN-based methods get terrible results because the unbalanced sample distribution forces the model to fit the distribution of rich-sample classes. Though CLB and MAEST obtain better results by exploiting information from unlabeled samples with self-supervised learning, the performance of the proposed TMAC is promoted further by

[1]Registered trademark.
[2]Trademarked.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 07:21:48 UTC from IEEE Xplore. Restrictions apply.

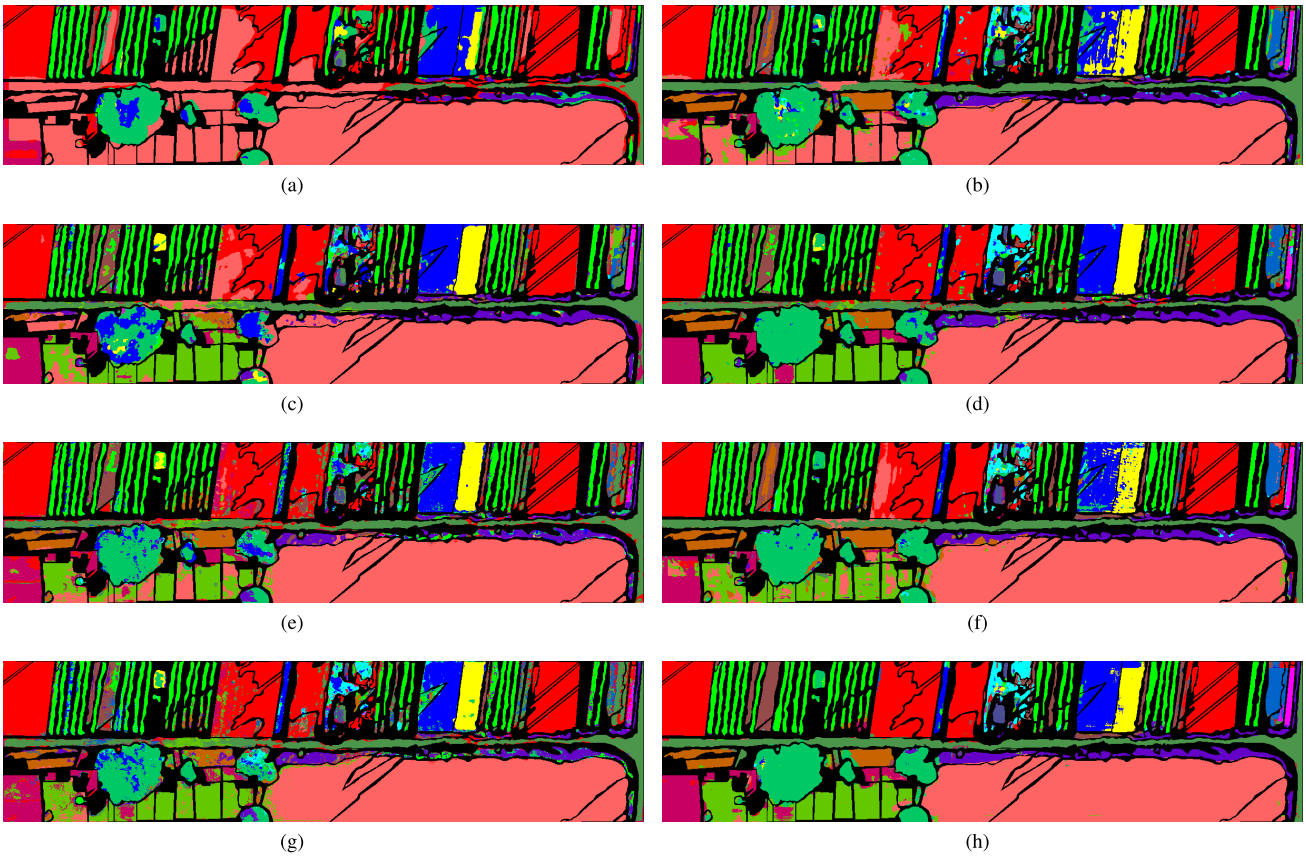


Fig. 8. Classification maps of HC dataset. (a) SVM. (b) ResNet. (c) HybridSN. (d) DBDA. (e) SF. (f) CLB. (g) MAEST. (h) TMAC.

combining the CL and MAE methods. Fig. 7 shows the visualization of the classification map of IP dataset. Our model shows much smoother classification results compared with other methods.

Table V listed the classification results of HC dataset. It can be seen that the proposed model TMAC still gets the highest value with three evaluation indexes. Because of the high complexity of the HC dataset, it can be seen that the classification result is not good for CNN-based methods. Also, for each category's accuracy, TMAC gets the best result for ten classes out of all 16 classes, which shows the strong classification ability of the proposed TMAC model. Fig. 8 shows the visualization classification results of HC dataset.

Table VI lists the classification results of PU dataset. Our proposed model TMAC still gets the best score on OA, AA, and Kappa. Its OA is 3.19% higher than that of the DBDA method, which shows the effect of the pretraining stage. For the class Bitumen whose class number is 7, methods, such as SVM, HybridSN, and DBDA, get poor results on it because the labeled training samples are not enough. The two self-supervised learning-based methods, CLB and MAEST, also obtained very different classification results. The proposed TMAC model still gets 99.16% accuracy on this class. It can be seen that our model has a great ability to deal with unbalanced data distribution. Fig. 9 shows the visualization of the classification map of these methods on the PU dataset. It can be seen that even some small isolated parts can be correctly classified by the TMAC model.

TABLE VII
INFLUENCE OF PRETRAINING STAGE

Dataset	Pretrained	Un-Pretrained
Indian Pine	97.28	90.80
HanChuan	95.41	89.51
Pavia University	96.89	93.39

TABLE VIII
PARAMETER ANALYSIS OF λ_f ON THE IP DATASET

Metrics	Loss weight			
	0.1	0.5	1	2
OA(%)	97.04	97.28	96.76	95.92
AA(%)	89.54	91.50	86.17	87.37
κ	96.63	96.89	96.31	95.35

D. Parameter Analysis

To give a more detailed evaluation of the proposed TMAC model, more results are presented to show the influence of some important parameters.

Table VII shows the effect of pretraining stage. The two models have the same parameter quantity and the same structure and use the same training data. The only difference is that TMAC has an unsupervised pretraining stage and the compared model is just trained 300 epochs in a supervised way. It can be seen that our TMAC has better performance over three datasets by taking advantage of the information of abundant unlabeled samples.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 07:21:48 UTC from IEEE Xplore. Restrictions apply.

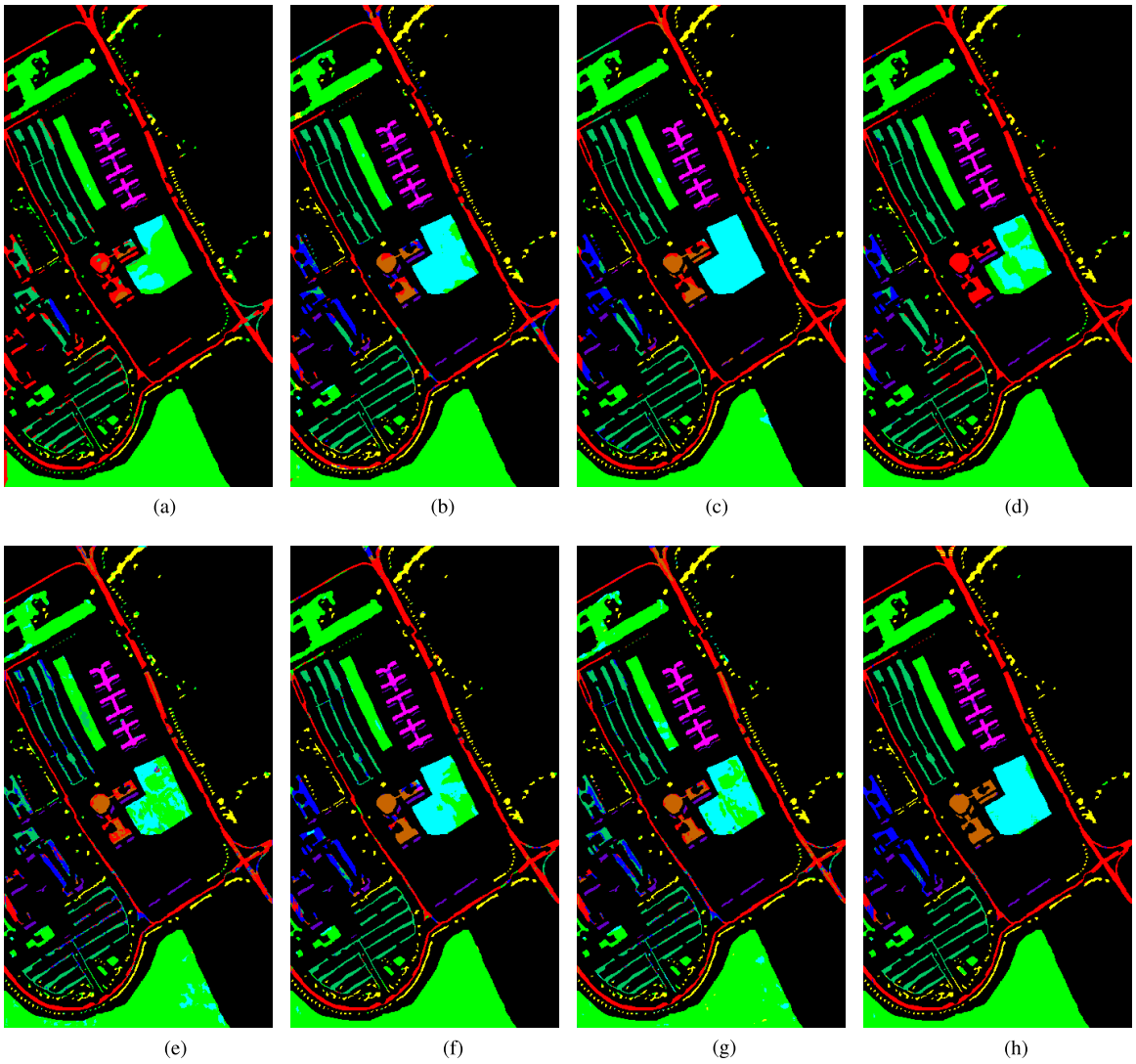


Fig. 9. Classification maps of PU. (a) SVM. (b) ResNet. (c) HybridSN. (d) DBDA. (e) SF. (f) CLB. (g) MAEST. (h) TMAC.

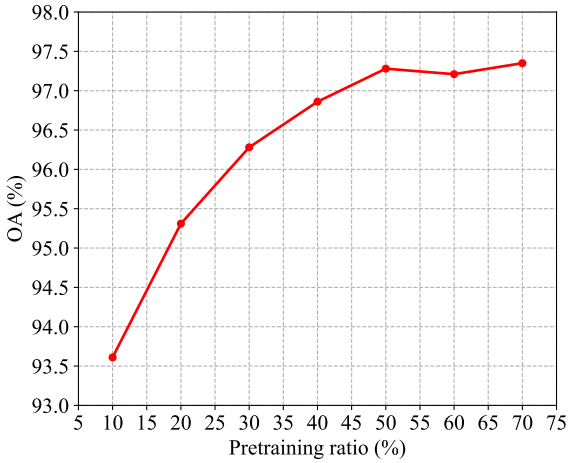


Fig. 10. Classification accuracy of TMAC with different pretraining ratios on the IP dataset.

Fig. 10 shows the influence of different pretraining ratios on the first stage. We use a pretraining ratio in the range of $[0.1, 0.7]$. It can be seen that when the ratio is in a low range, the increase in pretraining rate shows a great impact on the classification ability of the model, but when the ratio is up to almost 50%, the change in pretraining ratio has much less influence on the result. This is because when the ratio is low, the model does not have enough samples to learn the feature distribution. However, when the pretraining rate reaches a level, e.g., 50%, the sample is enough for the model to learn the feature distribution and more samples will bring much less benefit.

TABLE IX
PARAMETER ANALYSIS OF PATCH SIZE ON THE IP DATASET

Metrics	Patch size			
	9	15	21	27
OA(%)	77.72	88.70	96.09	97.28
AA(%)	65.99	77.28	89.23	91.50
κ	74.55	87.07	95.54	96.89

Table IX shows the influence of different patch sizes. For each pixel, we sampled an image patch with size $s \times s$ in its neighborhood. Using different patch sizes has discernible

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 07:21:48 UTC from IEEE Xplore. Restrictions apply.

TABLE X
PARAMETER ANALYSIS OF ENCODER LAYER ON THE IP DATASET

Metrics	Encoder layer			
	2	4	6	8
OA(%)	96.58	96.70	97.28	96.85
AA(%)	90.30	90.24	91.50	89.55
κ	96.10	96.24	96.89	96.40

TABLE XI
PARAMETER ANALYSIS OF DECODER LAYER ON THE IP DATASET

Metrics	Decoder layer			
	1	2	4	6
OA(%)	96.69	97.28	96.96	96.67
AA(%)	92.97	91.50	90.87	94.15
κ	96.24	96.89	96.54	96.20

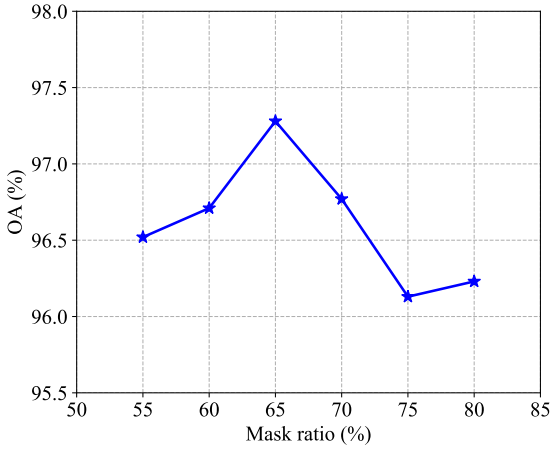


Fig. 11. Influence of different mask ratios on the IP dataset.

effects on the classification result. For the experiments, four patch sizes are used and the results are shown in Table IX. It can be seen that the increase in the patch size yields a corresponding enhancement in the accuracy. This is because, if the patch size goes up, the spectral–spatial information of the whole patch also increases. Besides, we think that this also benefits from the Transformer's long sequence modeling capability, which means that as the input sequence grows, the Transformer can learn more useful information from the data.

Tables X and XI show the influence of different encoder layers and decoder layers on the model's performance. The depth of encoder has an extreme effect on the model's feature extraction ability and the model's computational complexity. As shown in the Table X, a six-layer encoder gets the highest score. Also, for the decoder layer, a two-layer depth is enough for the model.

We also explore the influence of different mask ratios. Fig. 11 shows the interesting result that at first when the mask ratio goes up, the accuracy also increases accordingly, but when the mask ratio reaches 65%, the increase in mask ratio will negatively impact the performance. It may be because at first, by masking image, the encoder could learn a deep representation of the HSI data, but when the masked ratio exceeds the level, too much information is lost and the model cannot catch features correctly.

V. CONCLUSION

In this article, we proposed a new self-supervised learning model—TMAC. This method explored a new way of self-supervised learning for hyperspectral image classification. The algorithm is executed in a two-stage way. In the first stage, by combining MLM and CL, we force the model to focus on both pixelwise and holistic spatial–spectral features, which makes TMAC a strong feature extractor for hyperspectral images. Also, in the second stage, the downstream task classification is applied to the model, and by retaining the parameters in the first stage, we can just use a tiny fraction of labeled data to fine-tune the model. TMAC could get great accuracy on hyperspectral image datasets. Transformer also shows high potential in the hyperspectral image field by relying on its powerful ability to capture long-term dependencies, which is quite suitable for hyperspectral images because of the abundant bands.

The Transformer-based methods now have been the most popular and useful structure in the research. In the future, we will continue to explore the Transformer-based structure for HSI classification. In the meantime, how to reduce the compute complexity is an urgent task for the Transformer models. We will try to design the lightweight Transformer model that still has the high performance. On the other hand, multimodal deep learning has caught a lot of attention in recent years. Benefiting from its powerful ability to capture long-term dependencies, Transformer can easily catch the potential association between the different models. So how to use the Transformer in multimodal processing is a promising direction.

REFERENCES

[1] J. M. Bioucas-Dias, A. Plaza, G. Camps-Valls, P. Scheunders, N. Nasrabadi, and J. Chanussot, "Hyperspectral remote sensing data analysis and future challenges," *IEEE Geosci. Remote Sens. Mag. Replaces Newsletter*, vol. 1, no. 2, pp. 6–36, Jun. 2013.

[2] C. M. Gevaert, J. Suomalainen, J. Tang, and L. Kooistra, "Generation of spectral–temporal response surfaces by combining multispectral satellite and hyperspectral UAV imagery for precision agriculture applications," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 8, no. 6, pp. 3140–3146, Jun. 2015.

[3] X. Briottet et al., "Military applications of hyperspectral imagery," *Proc. SPIE*, vol. 6239, pp. 82–89, May 2006.

[4] Y.-Z. Feng and D.-W. Sun, "Application of hyperspectral imaging in food safety inspection and control: A review," *Crit. Rev. Food Sci. Nutrition*, vol. 52, no. 11, pp. 1039–1058, Nov. 2012.

[5] J. Xia, P. Ghamisi, N. Yokoya, and A. Iwasaki, "Random forest ensembles and extended multiextinction profiles for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 1, pp. 202–216, Jan. 2017.

[6] B. Guo, S. R. Gunn, R. I. Damper, and J. D. B. Nelson, "Customizing kernel functions for SVM-based hyperspectral image classification," *IEEE Trans. Image Process.*, vol. 17, no. 4, pp. 622–629, Apr. 2008.

[7] F. Luo, L. Zhang, X. Zhou, T. Guo, Y. Cheng, and T. Yin, "Sparse-adaptive hypergraph discriminant analysis for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 17, no. 6, pp. 1082–1086, Jun. 2020.

[8] C. Pan, X. Jia, J. Li, and X. Gao, "Adaptive edge preserving maps in Markov random fields for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 10, pp. 8568–8583, Oct. 2021.

[9] X. Kang, X. Xiang, S. Li, and J. A. Benediktsson, "PCA-based edge-preserving features for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 12, pp. 7140–7151, Dec. 2017.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 07:21:48 UTC from IEEE Xplore. Restrictions apply.

[10] J. Wang and C.-I. Chang, "Independent component analysis-based dimensionality reduction with applications in hyperspectral image analysis," *IEEE Trans. Geosci. Remote Sens.*, vol. 44, no. 6, pp. 1586–1600, Jun. 2006.

[11] L. Fang, N. He, S. Li, P. Ghamisi, and J. A. Benediktsson, "Extinction profiles fusion for hyperspectral images classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 3, pp. 1803–1815, Mar. 2018.

[12] H. Lee and H. Kwon, "Going deeper with contextual CNN for hyperspectral image classification," *IEEE Trans. Image Process.*, vol. 26, no. 10, pp. 4843–4855, Oct. 2017.

[13] L. Zhou, Y. Ye, T. Tang, K. Nan, and Y. Qin, "Robust matching for SAR and optical images using multiscale convolutional gradient features," *IEEE Geosci. Remote Sens. Lett.*, vol. 19, 2022, Art. no. 4017605.

[14] J. Cai, F. Gan, X. Cao, and W. Liu, "Signal modulation classification based on the transformer network," *IEEE Trans. Cognit. Commun. Netw.*, vol. 8, no. 3, pp. 1348–1357, Sep. 2022.

[15] C. Yu, R. Han, M. Song, C. Liu, and C.-I. Chang, "A simplified 2D-3D CNN architecture for hyperspectral image classification based on spatial–spectral fusion," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 13, pp. 2485–2501, 2020.

[16] Z. Zhong, J. Li, Z. Luo, and M. Chapman, "Spectral–spatial residual network for hyperspectral image classification: A 3-D deep learning framework," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 2, pp. 847–858, Feb. 2018.

[17] L. Mou, P. Ghamisi, and X. X. Zhu, "Deep recurrent neural networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 7, pp. 3639–3655, Jul. 2017.

[18] R. Hang, Q. Liu, D. Hong, and P. Ghamisi, "Cascaded recurrent neural networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 8, pp. 5384–5394, Aug. 2019.

[19] D. Hong, L. Gao, J. Yao, B. Zhang, A. Plaza, and J. Chanussot, "Graph convolutional networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 7, pp. 5966–5978, Jul. 2021.

[20] D. Hong et al., "More diverse means better: Multimodal deep learning meets remote-sensing imagery classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 5, pp. 4340–4354, May 2021.

[21] X. Wu, D. Hong, and J. Chanussot, "Convolutional neural networks for multimodal remote sensing data classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5517010.

[22] A. A. Aleissaee et al., "Transformers in remote sensing: A survey," 2022, *arXiv:2209.01206*.

[23] X. He, Y. Chen, and Z. Lin, "Spatial–spectral transformer for hyperspectral image classification," *Remote Sens.*, vol. 13, no. 3, p. 498, Jan. 2021.

[24] Z. Zhong, Y. Li, L. Ma, J. Li, and W.-S. Zheng, "Spectral–spatial transformer network for hyperspectral image classification: A factorized architecture search framework," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5514715.

[25] Y. Peng, Y. Zhang, B. Tu, Q. Li, and W. Li, "Spatial–spectral transformer with cross-attention for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5537415.

[26] B. Tu, X. Liao, Q. Li, Y. Peng, and A. Plaza, "Local semantic feature aggregation-based transformer for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5536115.

[27] Y. Ye, B. Zhu, T. Tang, C. Yang, Q. Xu, and G. Zhang, "A robust multimodal remote sensing image registration method and system using steerable filters with first- and second-order gradients," *ISPRS J. Photogramm. Remote Sens.*, vol. 188, pp. 331–350, Jun. 2022.

[28] S. S. Sawant and M. Prabukumar, "A review on graph-based semi-supervised learning methods for hyperspectral image classification," *Egyptian J. Remote Sens. Space Sci.*, vol. 23, no. 2, pp. 243–248, Aug. 2020.

[29] Y. Wu, G. Mu, C. Qin, Q. Miao, W. Ma, and X. Zhang, "Semi-supervised hyperspectral image classification via spatial-regulated self-training," *Remote Sens.*, vol. 12, no. 1, p. 159, Jan. 2020.

[30] B. Liu, X. Yu, P. Zhang, X. Tan, A. Yu, and Z. Xue, "A semi-supervised convolutional neural network for hyperspectral image classification," *Remote Sens. Lett.*, vol. 8, no. 9, pp. 839–848, Sep. 2017.

[31] L. Zhu, Y. Chen, P. Ghamisi, and J. A. Benediktsson, "Generative adversarial networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 9, pp. 5046–5063, Sep. 2018.

[32] C. Zhang, J. Yue, and Q. Qin, "Global prototypical network for few-shot hyperspectral image classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 13, pp. 4748–4759, 2020.

[33] H. Zhang, Y. Li, Y. Jiang, P. Wang, Q. Shen, and C. Shen, "Hyperspectral classification based on lightweight 3-D-CNN with transfer learning," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 8, pp. 5813–5828, Aug. 2019.

[34] A. van den Oord, Y. Li, and O. Vinyals, "Representation learning with contrastive predictive coding," 2018, *arXiv:1807.03748*.

[35] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, "Masked autoencoders are scalable vision learners," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 16000–16009.

[36] S. Hou, H. Shi, X. Cao, X. Zhang, and L. Jiao, "Hyperspectral imagery classification based on contrastive learning," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5521213.

[37] D. Ibanez, R. Fernandez-Beltran, F. Pla, and N. Yokoya, "Masked auto-encoding spectral–spatial transformer for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5542614.

[38] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 1–11.

[39] A. Dosovitskiy et al., "An image is worth 16×16 words: Transformers for image recognition at scale," 2020, *arXiv:2010.11929*.

[40] Y. Qing, W. Liu, L. Feng, and W. Gao, "Improved transformer net for hyperspectral image classification," *Remote Sens.*, vol. 13, no. 11, p. 2216, Jun. 2021.

[41] L. Sun, G. Zhao, Y. Zheng, and Z. Wu, "Spectral–spatial feature tokenization transformer for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5522214.

[42] D. Hong et al., "SpectralFormer: Rethinking hyperspectral image classification with transformers," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5518615.

[43] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, "Momentum contrast for unsupervised visual representation learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 9729–9738.

[44] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 1597–1607.

[45] J.-B. Grill et al., "Bootstrap your own latent-a new approach to self-supervised learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 21271–21284.

[46] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," 2018, *arXiv:1810.04805*.

[47] Z. Huang et al., "Contrastive masked autoencoders are stronger vision learners," 2022, *arXiv:2207.13532*.

[48] H. Bao, L. Dong, S. Piao, and F. Wei, "BEiT: BERT pre-training of image transformers," 2021, *arXiv:2106.08254*.

[49] C. Tao, H. Pan, Y. Li, and Z. Zou, "Unsupervised spectral–spatial feature learning with stacked sparse autoencoder for hyperspectral imagery classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 12, no. 12, pp. 2438–2442, Dec. 2015.

[50] X. Chen, S. Xie, and K. He, "An empirical study of training self-supervised vision transformers," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, Oct. 2021, pp. 9640–9649.

[51] Y. Zhong, X. Hu, C. Luo, X. Wang, J. Zhao, and L. Zhang, "WHU-hi: UAV-borne hyperspectral with high spatial resolution (H^2) benchmark datasets and classifier for precise crop identification based on deep convolutional neural network with CRF," *Remote Sens. Environ.*, vol. 250, Dec. 2020, Art. no. 112012.

[52] Y. Zhong et al., "Mini-UAV-borne hyperspectral remote sensing: From observation and processing to applications," *IEEE Geosci. Remote Sens. Mag. Replaces Newsletter*, vol. 6, no. 4, pp. 46–62, Dec. 2018.

[53] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.

[54] S. K. Roy, G. Krishna, S. R. Dubey, and B. B. Chaudhuri, "HybridSN: Exploring 3-D–2-D CNN feature hierarchy for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 17, no. 2, pp. 277–281, Feb. 2020.

[55] R. Li, S. Zheng, C. Duan, Y. Yang, and X. Wang, "Classification of hyperspectral image based on double-branch dual-attention mechanism network," *Remote Sens.*, vol. 12, no. 3, p. 582, Feb. 2020.

[56] X. Hu, T. Li, T. Zhou, Y. Liu, and Y. Peng, "Contrastive learning based on transformer for hyperspectral image classification," *Appl. Sci.*, vol. 11, no. 18, p. 8670, Sep. 2021.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 07:21:48 UTC from IEEE Xplore. Restrictions apply.